Counting, Carrying, and Circuits

Week 13 Tuesday — Boolean algebra, and how a computer adds and multiplies

COMSC 2043 • Discrete Structures and Critical Thinking

November 10, 2026

Today's destination

- Notice what **carrying** really is, using the decimal addition you already know.
- See that any set of symbols can be a number system (circles, stars, faces, fish).
- Add and multiply in **binary**, and see it is the same method as the one you learned.
- Meet Boolean algebra and build a **half adder** and a **full adder** from it.
- See why adding 16 bits fast needs a cleverer design: **ripple delay** versus **carry lookahead**.

Warm-up: what happens at $9 + 1$?

Count on your fingers or on paper, out loud if you like:

$$7 + 1 \quad 8 + 1 \quad 9 + 1 \quad 99 + 1$$

- What did you *run out of* at $9 + 1$?
- Where did the extra digit come from?

Counting from 1 to 10 in decimal

0 1 2 3 4 5 6 7 8 9 **10**

- We have **ten** symbols: 0 through 9. Not 10 symbols plus "10".
- At the end of the symbols, we **reset the ones place to 0** and **add 1 to the next place**.
- The "10" is two digits: a 1 in the tens place and a 0 in the ones place.

Add one column at a time: $52 + 53$

place	100s	10s	1s
carry	1		
		5	2
$+$		5	3
	1	0	5

1. **Ones:** $2 + 3 = 5$.
2. **Tens:** $5 + 5 = 10$. Write 0, **carry 1**.
3. **Hundreds:** nothing to add, so the carry drops down: 1.

Do one column, then work sideways to the next.

The same move at $99 + 1$

place	100s	10s	1s
carry	1	1	
		9	9
+			1
	1	0	0

1. $9 + 1 = 10$: write 0, carry 1.
2. $9 + 1$ (the carry) $= 10$: write 0, carry 1.
3. Nothing left to add, so the carry becomes a new digit.

Place values: ones $= 10^0$, tens $= 10^1$, hundreds $= 10^2$. **A new column appears every time the last one fills.**

Try the pattern: $67 + 58$

place	100s	10s	1s
carry	1	1	
		6	7
$+$		5	8
	1	2	5

1. **Ones:** $7 + 8 = 15$. Write 5, **carry 1**.
2. **Tens:** $6 + 5 + 1 = 12$. Write 2, **carry 1**.
3. **Hundreds:** the carry drops down: 1.

Say each column out loud, then work sideways.

Another one: $58 + 27$

place	100s	10s	1s
carry		1	
		5	8
$+$		2	7
		8	5

1. **Ones:** $8 + 7 = 15$. Write 5, **carry 1**.
2. **Tens:** $5 + 2 + 1 = 8$. No carry, so we are done.

Say each column out loud, then work sideways.

$999 + 1$

place	1000s	100s	10s	1s
carry	1	1	1	
		9	9	9
$+$				1
	1	0	0	0

1. Every column is $9 + 1 = 10$ or $9 + 0 + 1 = 10$: write 0, carry 1.
2. Three carries in a row. The last carry becomes a new digit.

Say each column out loud, then work sideways.

Octal: only eight symbols

Digits 0 1 2 3 4 5 6 7. There is no 8, no 9.

0 1 2 3 4 5 6 7 **10** 11

- At $7 + 1$ we run out of symbols, so we write 10 (that number is *eight*).
- Same pattern at the next column: 77 + 1 = 100 (that number is *sixty-four*).
- Place values are powers of 8: 1, 8, 64, 512, ...

Hexadecimal: sixteen symbols

Digits `0 1 2 3 4 5 6 7 8 9 A B C D E F`. We borrow letters because we need six more symbols.

8 9 A B C D E F **10** 11

- `9 + 7 = 10`: sixteen, so a one and a zero. `A + 5 = F`.
- `F + 1 = 10`. `FF + 1 = 100` (that number is two hundred fifty-six).
- Place values: 1, 16, 256, 4096, ...

The pattern, in every base

In base b there are b symbols, the biggest is $b - 1$.

	biggest digit $+1$	**biggest two digits** $+1$
base 10	$9 + 1 = 10$	$99 + 1 = 100$
base 8	$7 + 1 = 10$	$77 + 1 = 100$
base 16	$F + 1 = 10$	$FF + 1 = 100$

"10" always means one group of b and none left over. Only the name of the number changes.

We do not need digits at all

Four symbols: ○ ☆ ☺  (circle, star, face, fish) Agree on an order: ○ < ☆ < ☺ <  .

That is a base-4 system.

- ○ plays the role of 0, ☆ of 1, ☺ of 2,  of 3.
- After  , we are out of symbols: next is ☆○ ("one group of four, none left").

Counting 0 to 8 with circles, stars, faces and fish

decimal	base 4	symbols
0	0	○
1	1	☆
2	2	☺
3	3	
4	10	☆○
5	11	☆☆
6	12	☆☺
7	13	☆ 
8	20	☺○

Counting 9 to 16

decimal	base 4	symbols
9	21	
10	22	
11	23	
12	30	
13	31	
14	32	
15	33	
16	100	

Row 16 is the ☆○○ moment: the same thing as $99 + 1 = 100$.

Add in the invented system



Translate to digits to check: `3 + 1 = 10` in base 4, which is ☆◯. `3 + 3 = 12` in base 4, which is ☆☺.

- Write the answer in the symbols, not the digits. The arithmetic does not care what the marks look like.

Two symbols: binary

Only 0 and 1. Biggest digit is 1.

n	binary
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000

n	binary
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111
16	10000

Place values: $1, 2, 4, 8, 16, \ldots$

The first column fills at 1. So 1 + 1 = 10 (that is two).

Binary to decimal: write the place values above the bits

Write each place value above its bit.

place	64	32	16	8	4	2	1
	1	0	0	0	1	0	1

Circle the places that have a 1, then add: $64 + 4 + 1 = 69$.

place	128	64	32	16	8	4	2	1
	1	0	0	1	1	1	0	1

$128 + 16 + 8 + 4 + 1 = 157$.

Seven digits means the top place is $2^6 = 64$: places count from 0, so 2^0 is the first. Leading zeros change nothing: `0001` and `1` are both 1.

Decimal to binary, method 1: take the biggest power that fits

Convert 69. Go down the place values; write 1 if it fits, subtract, otherwise 0.

place	**bit**	**left over**
64	1	$69 - 64 = 5$
32	0	5 (too small)
16	0	5
8	0	5
4	1	$5 - 4 = 1$
2	0	1
1	1	$1 - 1 = 0$

Read the bits down: 1000101. Check: $64 + 4 + 1 = 69$.

Decimal to binary, method 2: divide by 2, read the remainders upward

$69 \div 2 = 34$ remainder $\mathbf{1}$
$34 \div 2 = 17$ remainder $\mathbf{0}$
$17 \div 2 = 8$ remainder $\mathbf{1}$
$8 \div 2 = 4$ remainder $\mathbf{0}$
$4 \div 2 = 2$ remainder $\mathbf{0}$
$2 \div 2 = 1$ remainder $\mathbf{0}$
$1 \div 2 = 0$ remainder $\mathbf{1}$

Stop when the quotient is 0. The only possible remainders are 0 and 1.

Read the remainders from the **bottom up**: 1000101.

Same answer as method 1. Check: $64 + 4 + 1 = 69$.

Binary addition: the whole rule table

sum	result
$0+0$	0
$0+1$	1
$1+0$	1
$1+1$	10 (write 0, carry 1)

That is the entire addition table. With a carry in, the last case can reach $1+1+1=11$ (write 1, carry 1).

Binary addition: one column at a time

place	16s	8s	4s	2s	1s
carry	1	1	1	0	
		1	0	1	1
+		0	1	1	0
	1	0	0	0	1

1. **1s:** $1 + 0 = 1$.
2. **2s:** $1 + 1 = 10$: write 0, carry 1.
3. **4s:** $0 + 1 + 1 = 10$: write 0, carry 1.
4. **8s:** $1 + 0 + 1 = 10$: write 0, carry 1.
5. **16s:** the carry drops down: 1.

$1011 + 0110 = 10001$, which is $11 + 6 = 17$. Smaller examples: $11 + 1 = 100$ $(3 + 1 = 4)$ and $101 + 011 = 1000$ $(5 + 3 = 8)$.

Another: $14 + 22 = 36$

place	32s	16s	8s	4s	2s	1s
carry	1	1	1	1	0	
		0	1	1	1	0
$+$		1	0	1	1	0
	1	0	0	1	0	0

Two five-digit numbers, added the way students picked them in class. Each step is still $0 + 0$, $0 + 1$, or $1 + 1 = 10$ (with a carry in, $1 + 1 + 1 = 11$).

$01110 = 14$, $10110 = 22$, $100100 = 32 + 4 = 36$.

Check: your turn

Add in binary, then convert both sides back to decimal to check.

`0111 + 0101` `1111 + 0001`

One sentence: **where did each carry come from?**

Binary multiplication: the one-digit table

product	result
0×0	0
0×1	0
1×0	0
1×1	1

A product of single bits is **1 only if both are 1**. Hold that thought: it is AND.

One bit times a number: copy it or zero it

Multiplying by 1 gives the number back. Multiplying by 0 gives zero. In decimal: $128 \times 1 = 128$ and $128 \times 0 = 0$.

Binary works the same: $1101 \times 1 = 1101$ and $1101 \times 0 = 0000$. That is why a row of zeros appears in a long multiplication: that bit of the bottom number was 0.

Binary long multiplication: same as the method you know

Decimal: $23 \times 14 = 322$

```
   23
x  14
-----
   92    (23 x 4)
  230    (23 x 10)
-----
  322
```

Binary: $5 \times 3 = 15$

```
   101
x   11
------
   101    (101 x 1)
  1010    (101 x 1, shifted)
------
  1111
```

Each partial product is either the top number or zeros, then shifted. Then **add**, using the rule table.

More binary multiplication

```
   11
x  10
-----
   00
  110
-----
  110
```

$3 \times 2 = 6$

```
    110
x   101
------
    110
   0000
  11000
------
  11110
```

$6 \times 5 = 30$

Where does a computer do this? Boolean algebra

Boolean algebra has two values, true/false, which we write 1/0, and three basic operations:

AND $(A \cdot B)$

A	B	out
0	0	0
0	1	0
1	0	0
1	1	1

OR $(A + B)$

A	B	out
0	0	0
0	1	1
1	0	1
1	1	1

NOT $(\overline{A})$

A	out
0	1
1	0

A trap: Boolean $1 + 1$ is not binary $1 + 1$

In Boolean algebra the plus sign means OR:

$$1 + 1 = 1$$

"true or true is true."

In binary arithmetic the plus sign means add:

$$1 + 1 = 10$$

one, one: write 0, carry 1.

Same marks, different rules. **Always say which system you are in** (Sources, Rules/Assumptions).

A fourth gate: XOR (exclusive or)

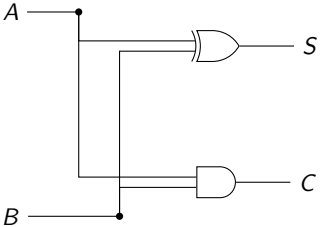
XOR is true when the inputs **differ**. Written $A \oplus B$.

A	B	**XOR**	**binary** $A + B$
0	0	0	00
0	1	1	01
1	0	1	01
1	1	0	10

Look at the last two columns: the **right-hand digit of the sum is the XOR**, and the **left-hand digit (the carry) is the AND**.

The half adder

Adds two bits A and B. Outputs: sum $S = A \oplus B$, carry $C = A \cdot B$.



A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Read CS as a two-bit number: it equals $A + B$.

The problem a half adder cannot solve

Inside a long sum, every column after the first receives three bits: A, B, and the **carry-in** from the column to its right.

```
  1111      <- carries
   01101
 + 00111
 -------
  10100
```

That is three inputs (A, B, C_{in}) and two outputs (sum, carry-out). We need a **full adder**.

The full adder

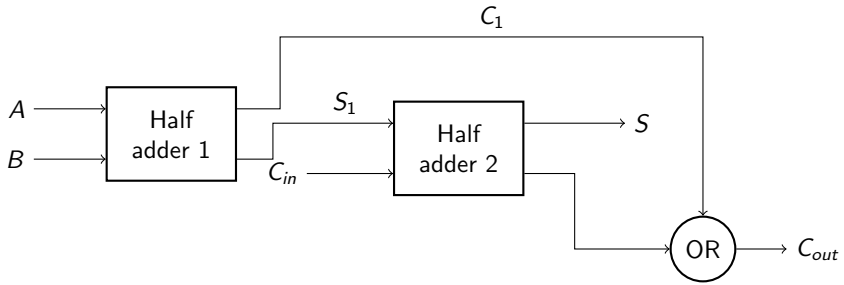
Sum $S = A \oplus B \oplus C_{in}$. Carry-out $C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$.

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Read $C_{out}S$ as a two-bit number: it equals $A + B + C_{in}$ in every row.

Eight rows is every possible input. **Eight rows is a proof.**

Full adder = two half adders and an OR



Half adder 1 adds $A + B$. **Half adder 2** adds that sum plus C_{in}. Either one can make a carry, so OR combines them.

Optional: the full adder as Karnaugh maps

Maurice Karnaugh (Bell Labs, 1953) arranged a truth table in a grid so patterns jump out. The columns go $00, 01, 11, 10$ (only one input changes between neighbors), *not* counting order.

C_{out}

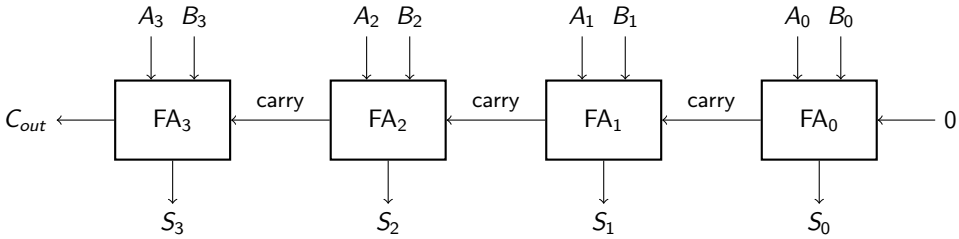
	AB			
C_{in}	00	01	11	10
0	0	0	1	0
1	0	1	1	1

S

	AB			
C_{in}	00	01	11	10
0	0	1	0	1
1	1	0	1	0

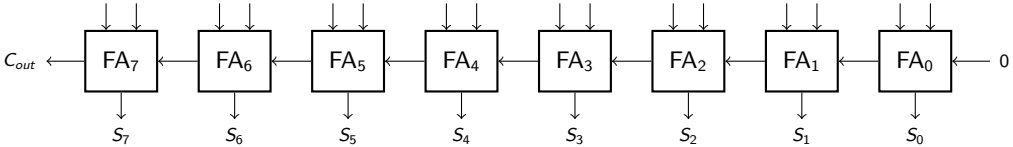
C_{out}: group the 1s in pairs: AB, $A\,C_{in}$, $B\,C_{in}$, so $C_{out} = AB + AC_{in} + BC_{in}$ (at least two inputs are 1).
S is a checkerboard: no pair of neighbors to merge, which is XOR.

Chain them: a 4-bit ripple-carry adder



Each carry ripples left, one stage at a time: the same ripple as $99 + 1 = 100$.

Scale up: an 8-bit ripple-carry adder



Worst case: 11111111 + 00000001 = 100000000 ($255 + 1 = 256$). The carry is created in the *first* column and must travel through **all eight**.

Delay: signals take time

A gate is a physical thing. Its output changes a tiny moment *after* its inputs do. Call that one **gate delay**.

- In one full adder, a carry-in passes through an AND, then an OR: **2 gate delays** per stage.
- The carry into bit i is ready after about $2i + 1$ gate delays; the sum bit needs one more XOR.

bits n	4	8	16	32	64
last sum bit ready after	8	16	32	64	128 gate delays

Delay grows in step with the number of bits: **about** $2n$. The slowest path sets how fast the clock may tick.

Can we get the carries sooner?

Each stage asks one of two questions about its two input bits:

Generate: $G_i = A_i \cdot B_i$
This column makes a carry *by itself* $(1 + 1)$.

Propagate: $P_i = A_i \oplus B_i$
This column *passes along* a carry that arrives (exactly one 1).

Carry out of a column: $C_{i+1} = G_i + P_i \cdot C_i$. Now **substitute** instead of waiting. The carries only depend on the inputs and C_0:

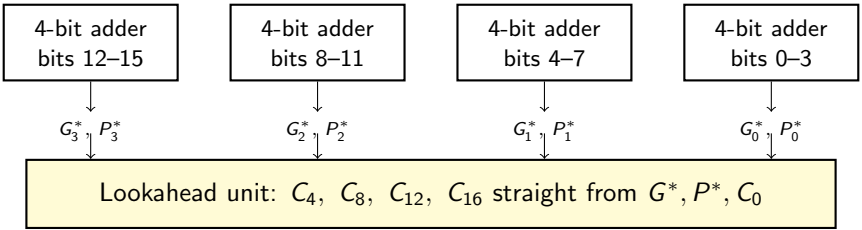
$$C_1 = G_0 + P_0 C_0$$
$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

A 4-bit carry-lookahead unit

$$
\begin{aligned}
C_1 &= G_0 + P_0 C_0 \\
C_2 &= G_1 + P_1 G_0 + P_1 P_0 C_0 \\
C_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \\
C_4 &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0
\end{aligned}
$$

Every carry is ready after **3 gate delays** (P and G: 1; the AND terms: 1; the OR: 1). Then sums: $S_i = P_i \oplus C_i$, **4 total**, against 8 for 4-bit ripple.

16 bits: lookahead on top of lookahead

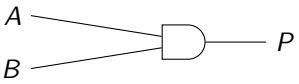


Each 4-bit block reports a **block generate** G^* and **block propagate** P^* ("all four propagate"). The same trick, applied to blocks.

	ripple (16)	**lookahead (16)**
last sum ready after	32	**8** gate delays

Multiplying bits: the 1-bit multiplier

A	B	$P = A \cdot B$	decimal
0	0	0	$0 \times 0 = 0$
0	1	0	$0 \times 1 = 0$
1	0	0	$1 \times 0 = 0$
1	1	1	$1 \times 1 = 1$



The product of two bits is 1 only when **both** are 1. That is exactly an **AND** gate. The 1-bit multiplier *is* one AND gate.

One bit times a whole number: one AND gate per bit

```
  1 1 0 1        1 1 0 1
x       1      x       0
---------      ---------
  1 1 0 1        0 0 0 0
```

$13 \times 1 = 13$ $\quad$ $13 \times 0 = 0$

Each bit of the top number is ANDed with the one multiplier bit B:

$$P_i = A_i \cdot B$$

If $B = 1$ every bit passes through (a copy). If $B = 0$ every bit becomes 0. **One AND gate per bit.**

2-bit $\times$ 2-bit: line up the partial products

$$
\begin{array}{cccc}
 & & A_1 & A_0 \\
\times & & B_1 & B_0 \\
\hline
 & & A_1B_0 & A_0B_0 \\
 & A_1B_1 & A_0B_1 & \\
\hline
P_3 & P_2 & P_1 & P_0
\end{array}
$$

1. Row 1 is $A \times B_0$: two AND gates.
2. Row 2 is $A \times B_1$, shifted left one place: two more AND gates.
3. Add the columns, right to left, with carries.

That is 4 AND gates, then an adder for the middle columns.

2-bit $\times$ 2-bit: a worked example, $3 \times 3 = 9$

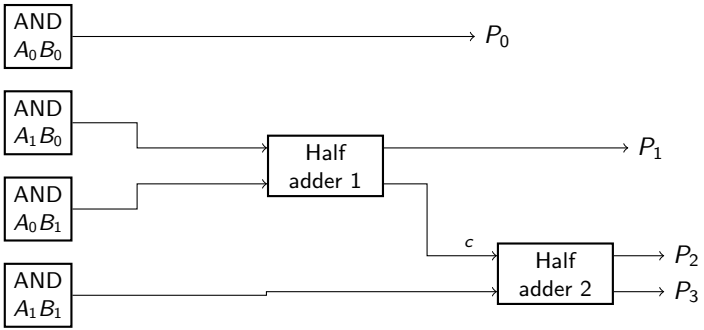
carry	1	1		
			1	1
$\times$			1	1
			1	1
		1	1	
	1	0	0	1

1. P_0: only one bit here, $1 \cdot 1 = 1$.
2. P_1: $1 + 1 = 10$. Write 0, carry 1.
3. P_2: 1 (from row 2) + the carry $= 10$. Write 0, carry 1.
4. P_3: the carry drops down: 1.

Answer $1001 = 9$ $(3 \times 3 = 9)$.

The 2-bit multiplier is two half adders

$$P_0 = A_0 B_0$$
$$P_1 = A_1 B_0 \oplus A_0 B_1 \qquad c = A_1 B_0 \cdot A_0 B_1 \quad \text{(half adder 1)}$$
$$P_2 = A_1 B_1 \oplus c \qquad P_3 = A_1 B_1 \cdot c \quad \text{(half adder 2)}$$



Truth table: the 2-bit multiplier

A_1A_0	B_1B_0	$P_3P_2P_1P_0$	decimal
00	00	0000	$0 \times 0 = 0$
00	01	0000	$0 \times 1 = 0$
00	10	0000	$0 \times 2 = 0$
00	11	0000	$0 \times 3 = 0$
01	00	0000	$1 \times 0 = 0$
01	01	0001	$1 \times 1 = 1$
01	10	0010	$1 \times 2 = 2$
01	11	0011	$1 \times 3 = 3$

A_1A_0	B_1B_0	$P_3P_2P_1P_0$	decimal
10	00	0000	$2 \times 0 = 0$
10	01	0010	$2 \times 1 = 2$
10	10	0100	$2 \times 2 = 4$
10	11	0110	$2 \times 3 = 6$
11	00	0000	$3 \times 0 = 0$
11	01	0011	$3 \times 1 = 3$
11	10	0110	$3 \times 2 = 6$
11	11	1001	$3 \times 3 = 9$

All 16 input rows, so this table is a proof for 2 bits.

4-bit $\times$ 4-bit: the same idea, more rows

```
      1 0 1 1     (11)
    x 1 1 0 1     (13)
    ---------
      1 0 1 1     11 x 1
    0 0 0 0       11 x 0, shifted 1
  1 0 1 1         11 x 1, shifted 2
1 0 1 1           11 x 1, shifted 3
---------------
1 0 0 0 1 1 1 1   (143)
```

Decimal check, both ways:
$1011 = 8 + 2 + 1 = 11$, $1101 = 8 + 4 + 1 = 13$.
$11 \times 13 = 143 = 128 + 8 + 4 + 2 + 1 = 10001111$.

4 rows of partial products, 16 AND gates in all. Then add the 4 rows, two at a time, with half and full adders.

It is modular: the same blocks, repeated

$n \times n$ bits	AND gates	half adders	full adders
1	1	0	0
2	4	2	0
4	16	4	8
8	64	8	48

- **AND gates** make every partial-product bit (n^2 of them).
- **Shifting is just wiring**: a row moves left by connecting its bits to different columns.
- **Half and full adders** add the rows, carry rippling along each row, just like the ripple-carry adder.

Same carry ripple, so the delay grows with the number of bits; fast multipliers use lookahead ideas too.

AI Fluency check

An AI says: "$A + B$ simplifies to A because it worked for $A = 1,\ B = 1$."

- Which system is the plus sign in: Boolean OR or binary addition?
- Is one satisfying row a proof? What would be?

Write one sentence: what does a complete truth table prove that a single example cannot?

Exit ticket and Thursday

Exit ticket (one sentence each):

1. In your own words: what does “carry” mean?
2. Which gate adds one bit, and which gate makes the carry?

Thursday: Show & Tell. **Next Tuesday:** finite-state machines (coin machine, and more) with Mermaid diagrams.